UNIT 3 · TOPIC 3.3

Network Segmentation

Assignment

Instructions: 9 practice questions + 2 coding assignment(s), 18 points total. Show your work for short-answer items. Coding assignments may be completed here or in the interactive auto-graded environment on the course site (same problem, same points).

Name: **Date:** **Class:**

Practice Questions

3.3.a · VLANs and network segmentation

1. A host on VLAN 10 wants to send data to a host on VLAN 20. What device must the traffic pass through? (1 pt)

- ○ A. A hub
- ○ B. A router or layer-3 switch
- ○ C. Another VLAN-tagged host
- ○ D. The same switch port

2. Which of the following are TRUE about VLANs? Select ALL that apply. (3 pts)

- □ A. VLANs are logical, not physical, separations
- □ B. A single switch can host multiple VLANs simultaneously
- □ C. VLANs prevent all forms of attack with no additional configuration
- □ D. Broadcast traffic on one VLAN is not visible to hosts on a different VLAN

(Select ALL that apply.)

3. An attacker compromises a guest device on the visitor VLAN. The corporate file server is on the server VLAN. Assuming correct VLAN configuration and restrictive firewall rules, what can the attacker reach from the guest device? (1 pt)

- ○ A. The file server and all other internal servers
- ○ B. Only trunk ports
- ○ C. Any device on any VLAN because they share the same switch
- ○ D. Only devices on the same guest VLAN; the file server is unreachable

4. What is the purpose of a trunk port? (1 pt)

- ○ A. To block all VLAN traffic
- ○ B. To carry 802.1Q-tagged frames for multiple VLANs on a single physical link
- ○ C. To connect an end-user workstation to exactly one VLAN
- ○ D. To assign IP addresses to hosts

5. Name the attack technique where an intruder who has compromised one host attempts to reach additional hosts deeper in the network. (1 pt)

3.3.b · Traffic isolation and zero-trust segmentation

6. What is the core paradigm of a Zero Trust Architecture (ZTA)? (1 pt)

- ○ A. Never trust, always verify, regardless of where the connection request originates
- ○ B. Implicit trust is granted once a device successfully authenticates to the VPN
- ○ C. Trust but verify internal hosts based on their physical network connection
- ○ D. Trust all devices connected to the internal Management VLAN

7. Which of the following are core practices of a Zero Trust Network Architecture? Select ALL that apply. (2 pts)

- □ A. Verifying device health and compliance posture before granting resource access
- □ B. Permitting unlimited lateral movement within the server VLAN
- □ C. Using microsegmentation to limit the blast radius of a breach
- □ D. Granting broad access privileges based on physical location in the office

(Select ALL that apply.)

8. In a Private VLAN (PVLAN) setup, what type of port is completely blocked from communicating with any other ports in the same PVLAN, except for promiscuous ports? (1 pt)

- ○ A. Community port
- ○ B. Isolated port
- ○ C. Trunk port
- ○ D. Secondary port

9. Name the security technique that applies fine-grained security policies to individual hosts or workloads, treating each device as its own isolated boundary to limit lateral movement. (1 pt)

Coding Assignments (Python 3)

3.3.a · VLANs and network segmentation

3 pts · auto-graded on the site

Coding problem 3.3.a: VLAN segmentation flow filter

Skill: Mitigate Risk · **Maps to:** 3.3.A · **Language:** Python 3 (pytest)

Problem

A network switch enforces VLAN segmentation using the following default-deny policy:

- Traffic between two hosts **on the same VLAN** is **allowed**.
- Traffic from VLAN 20 (Employee) to VLAN 30 (Server) is **allowed** (business policy exception).
- All other cross-VLAN traffic is **blocked**.

Given a mapping of host names to VLAN IDs and a list of traffic flows, return the list of flows that are **blocked** by these rules.

Function contract

Implement `solve(vlan_map: dict, flows: list) -> list` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `vlan_map`: `dict[str, int]`, maps each host name to its VLAN ID (integer).
- `flows`: `list[tuple[str, str]]`, each tuple is `(src_host, dst_host)`.

Output

A `list[tuple[str, str]]` of flows that are blocked, in the same order they appear in the input. Do not include allowed flows.

Examples

```
vlan_map = {"WS-01": 20, "SRV-01": 30, "GUEST-01": 40, "MGMT-01": 10}
flows = [("WS-01","SRV-01"), ("GUEST-01","SRV-01"), ("WS-01","WS-01")]

solve(vlan_map, flows)
-> [("GUEST-01","SRV-01")]
# WS-01->SRV-01: VLAN 20->30, allowed by business exception
# GUEST-01->SRV-01: VLAN 40->30, blocked (no exception)
# WS-01->WS-01: same VLAN, allowed
```

Constraints

- Deterministic, offline, standard library only.
- All host names in `flows` are guaranteed to be in `vlan_map`.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 3.3.a. Student completes the TODO.
This file intentionally FAILS the tests until implemented.
"""
from typing import List, Tuple, Dict


def solve(vlan_map: Dict[str, int], flows: List[Tuple[str, str]]) -> List[Tuple[str, str]]:
    """Determine which traffic flows are blocked by VLAN segmentation rules.

    Args:
        vlan_map: dict mapping host name to its VLAN ID.
        flows: list of tuples (src_host, dst_host).

    Returns:
        List of blocked flows in the same order.
    """
    # TODO: implement per prompt.md
    raise NotImplementedError("implement solve() — see prompt.md")
```

3.3.b · Traffic isolation and zero-trust segmentation

3 pts · auto-graded on the site

Coding problem 3.3.b: Zero Trust Access Proxy

Skill: Mitigate Risk · **Maps to:** 3.3.B, 3.3.C · **Language:** Python 3 (pytest)

Problem

In a Zero Trust Network Architecture (ZTNA), network access is never implicitly granted based on a user's location. Instead, every request to access a corporate resource must be explicitly authenticated, authorized, and checked for health compliance.

You are tasked with implementing the evaluation logic for a Zero Trust access proxy. The proxy evaluates a batch of access requests against a central policy and filters out any requests that fail to meet authorization, location, or posture requirements.

Function contract

Implement `solve(requests: list, policy: dict) -> list` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `requests`: A list of dictionaries representing connection requests. ``python requests = [{ "id": "REQ-001", "user": "alice", "role": "doctor", "resource": "ehr_db", "ip": "192.168.1.50", "device_posture": {"antivirus_ok": True, "disk_encrypted": True} }]
- `policy`: A dictionary detailing the security rules. ``python policy = { "allowed_roles": { "ehr_db": ["doctor", "admin"], "mri_machine": ["admin", "technician"] }, "requires_internal": ["ehr_db", "mri_machine"], "requires_posture": ["ehr_db"], "internal_prefixes": ["192.168.1.", "10.0."] }

Output

A list of strings representing the `id`s of requests that are **allowed**, maintaining their original order.

Examples

```
requests = [
    {
        "id": "REQ-001",
        "user": "alice",
        "role": "doctor",
        "resource": "ehr_db",
        "ip": "192.168.1.50",
        "device_posture": {"antivirus_ok": True, "disk_encrypted": True}
    },
    {
        "id": "REQ-002",
        "user": "bob",
        "role": "guest",
        "resource": "ehr_db",
        "ip": "192.168.1.60",
        "device_posture": {"antivirus_ok": True}
    }
]
# REQ-001 is allowed (role is doctor, IP matches internal prefix, posture checks are all True).
# REQ-002 is denied (guest role is not allowed on ehr_db).

solve(requests, policy) -> ["REQ-001"]
```

Constraints

- Deterministic, offline, standard library only.
- All requests have the required keys.
- Must pass all cases in `tests.py`.

Starter code:

```
"""Starter code for 3.3.b. Student completes the TODO.
This file intentionally FAILS the tests until implemented.
"""
from typing import List, Dict, Any


def solve(requests: List[Dict[str, Any]], policy: Dict[str, Any]) -> List[str]:
    """Evaluate Zero Trust connection requests against access policy rules.

    Args:
        requests: list of request dictionaries.
        policy: policy dictionary.

    Returns:
        List of request IDs that are allowed.
    """
    # TODO: implement per prompt.md
    raise NotImplementedError("implement solve() — see prompt.md")
```